

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Объектно-ориентированное программирование»

Студент гр. 4384

Серженко Д.К.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2025

ЦЕЛЬ РАБОТЫ

Разработать архитектуру пошаговой стратегической игры с применением объектно-ориентированного подхода. Создать систему взаимодействующих игровых объектов (игрок, враги, здания) на клеточном поле с препятствиями и специальными клетками. Реализовать базовые игровые механики: перемещение, систему боя с двумя режимами атаки, спавн врагов. Освоить работу с динамической памятью через реализацию конструкторов копирования и перемещения для игрового поля. Закрепить принципы инкапсуляции и безопасного управления ресурсами в C++.

ЗАДАНИЕ

1. Создать класс(ы) игры, который реализует основной цикл игры, и которому передаются команды от пользователя. Игровой цикл состоит из следующих шагов:
 - a. Начало игры
 - b. Запуск уровня
 - c. Ход игрока. Ход, атака или применение заклинания.
 - d. Ход союзников - если имеются
 - e. Ход врагов
 - f. Ход вражеской базы и башни - если имеются
2. Условие прохождения уровня студент определяет самостоятельно. Если игрок проигрывает, то игроку должно предлагаться начать заново игру, либо выйти из программы. Все взаимодействие должно происходить через классы игры.
3. Реализовать систему сохранения и загрузки игры. Пользователь должен иметь возможность сохранить игру в любой момент. Пользователь должен иметь возможность загружаться при запуске программы (или выбрать новую), либо во время игры. Сохранения должны оставаться в консистентном состоянии между запусками игры.

Добавить обработку исключительных ситуаций для загрузки/сохранения, например, невозможность записать в файл, нельзя загрузиться так как файл не существует или в нем некорректные данные.

Примечания:

- Класс игры может знать о игровых сущностях, но не наоборот
- При работе с файлом используйте идиому RAII.
- Исключения должны обязательно обрабатываться, и программа не должна завершаться
- Исключения должны быть информативными (содержать информацию о том, что и где произошло), на разные виды исключительных ситуаций должны быть свои исключения

ВЫПОЛНЕНИЕ РАБОТЫ

1. Общая архитектура системы

Архитектура игры расширена добавлением системы управления игровым циклом, механизма сохранения/загрузки и обработки исключений.

Основные компоненты:

- `GameSession` — центральный класс, управляющий игровым процессом
- `TurnManager` — класс для управления последовательностью ходов
- `SaveManager` — класс для сохранения и загрузки игры (RAII)
- Иерархия исключений для обработки ошибок
- Обновленные классы `Player`, `Enemy`, `Field` с поддержкой сериализации

2. Иерархия классов и их назначение

1) Класс `GameSession`

Назначение: центральный класс, реализующий основной игровой цикл и управляющий всеми игровыми объектами.

```
class GameSession {
public:
    GameSession(int width, int height, int enemyCount);
```

```

void run(); // Основной игровой цикл

// Методы для сохранения/загрузки
const Field& getField() const;
const Player& getPlayer() const;
const std::vector<std::shared_ptr<Enemy>>& getEnemies() const;

void addEnemy(std::shared_ptr<Enemy> enemy);

private:
    Field field;
    Player player;
    std::vector<std::shared_ptr<Enemy>> enemies;
    std::vector<std::shared_ptr<ITarget>> allTargets;
    std::vector<std::unique_ptr<ISpell>> spellDeck;
    TurnManager turnManager;

    // Внутренние методы игрового цикла
    void processPlayerAction(const PlayerAction& action);
    void processEnemiesTurn();
    void render() const;
    void resetLevel();
    void resetGame();
    bool isLevelComplete() const;
};

```

Обоснование: класс инкапсулирует всю логику игрового процесса, обеспечивая четкое разделение ответственности и предотвращая появление "божественного" класса.

2) Класс TurnManager

Назначение: управление последовательностью ходов в игре.

```

class TurnManager {
public:
    enum class Phase {
        PLAYER_TURN,
        ENEMIES_TURN,
        LEVEL_COMPLETE,
        GAME_OVER
    };

    TurnManager();

    void nextPhase();
    bool isPlayerTurn() const;
    bool isEnemiesTurn() const;
    bool isLevelComplete() const;
    bool isGameOver() const;

    Phase getCurrentPhase() const;
    void setPhase(Phase phase);

    friend std::ostream& operator<<(std::ostream& os, const TurnManager&
tm);

```

```
private:
    Phase currentPhase;
};
```

Обоснование: выделение логики управления ходами в отдельный класс позволяет упростить основной игровой цикл и делает код более модульным и тестируемым.

3) Иерархия исключений

Назначение: специализированные классы для различных типов ошибок.

```
class GameException : public std::runtime_error {
public:
    explicit GameException(const std::string& message);
};

class SaveException : public GameException {
public:
    explicit SaveException(const std::string& message);
};

class LoadException : public GameException {
public:
    explicit LoadException(const std::string& message);
};

class CorruptedSaveException : public LoadException {
public:
    explicit CorruptedSaveException(const std::string& message);
};

class FileAccessException : public SaveException {
public:
    explicit FileAccessException(const std::string& filename);
};
```

Обоснование: иерархия исключений позволяет точно определять тип ошибки и предоставлять пользователю информативные сообщения. Разные типы исключений могут обрабатываться по-разному в зависимости от контекста.

4) Класс SaveManager (RAII)

Назначение: управление сохранением и загрузкой игры с применением идиомы RAII.

```
class SaveManager {
private:
    std::fstream file;
    std::string filename;

    // Запрет копирования
    SaveManager(const SaveManager&) = delete;
    SaveManager& operator=(const SaveManager&) = delete;

    // Методы сериализации
    void writeString(const std::string& str);
    void writeInt(int value);
    void writeBool(bool value);

    std::string readString();
    int readInt();
    bool readBool();

public:
    explicit SaveManager(const std::string& filename);
    ~SaveManager(); // RAII - автоматическое закрытие файла

    // Разрешено перемещение
    SaveManager(SaveManager&& other) noexcept;
    SaveManager& operator=(SaveManager&& other) noexcept;

    void saveGame(const GameSession& game);
    std::unique_ptr<GameSession> loadGame();

    static std::vector<std::string> listSaves();
};
```

Обоснование: применение идиомы RAII гарантирует, что файл будет правильно закрыт даже в случае возникновения исключения. Запрет

копирования предотвращает случайное дублирование файловых дескрипторов.

5) Структура PlayerAction

Назначение: унифицированное представление действия игрока.

```
struct PlayerAction {  
    enum Type { MOVE, CAST_SPELL, SAVE, LOAD, QUIT, INVALID } type;  
  
    char direction;  
    int spellIndex;  
    int targetX;  
    int targetY;  
    std::string filename;  
  
    PlayerAction() : type(INVALID), direction(0), spellIndex(-1),  
targetX(0), targetY(0) {}  
  
    static PlayerAction makeMove(char dir);  
    static PlayerAction makeCast(int index, int x, int y);  
    static PlayerAction makeSave(const std::string& fname);  
    static PlayerAction makeLoad(const std::string& fname);  
    static PlayerAction makeQuit();  
    static PlayerAction makeInvalid();  
};
```

Обоснование: использование структуры с фабричными методами позволяет единообразно обрабатывать все типы действий игрока и избежать сложных условных конструкций.

3. Система игрового цикла

1) Последовательность ходов

Игровой цикл реализован в методе `GameSession::run()` и состоит из следующих фаз:

- Отрисовка текущего состояния — вывод игрового поля и информации
- Ход игрока — ожидание и обработка ввода
- Ход врагов — автоматическое перемещение и атака врагов

- Проверка завершения уровня — если все враги уничтожены, переход на следующий уровень

- Проверка окончания игры — если игрок погиб, предложение начать заново

```
void GameSession::run() {
    while (!gameOver) {
        try {
            render();

            if (turnManager.isPlayerTurn()) {
                PlayerAction action = getPlayerAction();

                if (action.type != PlayerAction::INVALID) {
                    processPlayerAction(action);

                    if (action.type != PlayerAction::SAVE &&
                        action.type != PlayerAction::LOAD &&
                        action.type != PlayerAction::QUIT) {
                        turnManager.nextPhase();
                    }
                }
            }

            if (turnManager.isEnemiesTurn() && !gameOver) {
                processEnemiesTurn();
                turnManager.nextPhase();
            }

            if (isLevelComplete() && !gameOver) {
turnManager.setPhase(TurnManager::Phase::LEVEL_COMPLETE);
                resetLevel();
            }

            } catch (const GameException& e) {
                std::cerr << "Игровая ошибка: " << e.what() << std::endl;
            }
        }
    }
}
```

2) Обработка действий игрока

Действия игрока обрабатываются через унифицированный интерфейс:

- Перемещение (w/a/s/d) — проверка границ поля, атака при встрече с врагом

- Использование заклинания (c) — выбор заклинания из руки, проверка маны, применение

- Сохранение игры (p) — запись текущего состояния в файл

- Загрузка игры (l) — восстановление состояния из файла

- Помощь (h) — вывод подсказок
- Выход (q) — завершение игры

4. Система сохранения и загрузки

1) Формат сохранения

Сохранение выполняется в бинарном формате со следующей структурой:

[Заголовок] "GAME_SAVE_V1" + временная метка

[Поле] ширина, высота

[Игрок] здоровье, макс здоровье, урон, мана, макс мана, очки, x, y

[Рука] количество заклинаний, емкость

[Для каждого заклинания] имя, стоимость маны, дальность, тип, урон, размер области

[Враги] количество

[Для каждого врага] здоровье, урон, x, y, жив ли

[Статистика] побеждено врагов, игра окончена

2) Процесс сохранения

```
void SaveManager::saveGame(const GameSession& game) {
    file.open(filename,      std::ios::binary      |      std::ios::out      |
std::ios::trunc);
    if (!file.is_open()) {
        throw FileAccessException(filename);
    }

    try {
        writeHeader();

        // Запись всех игровых данных
        writeInt(game.getField().getWidth());
        writeInt(game.getField().getHeight());

        const Player& player = game.getPlayer();
```

```

        writeInt(player.getHealth());
        writeInt(player.getMaxHealth());
        writeInt(player.getPosition().getX());
        writeInt(player.getPosition().getY());
        // ... запись остальных данных

        file.close();
    } catch (...) {
        file.close();
        throw;
    }
}

```

3) Процесс загрузки

```

void GameSession::processLoad(const std::string& filename) {
    try {
        SaveManager saver(filename);
        auto loadedGame = saver.loadGame();

        if (loadedGame) {
            // Создаем новое пустое поле
            Field newField(loadedGame->field.getWidth(),
                           loadedGame->field.getHeight());

            // Очищаем текущие данные
            enemies.clear();
            spellDeck.clear();

            // Переносим данные из загруженной игры
            field = std::move(newField);
            player = std::move(loadedGame->player);
            enemies = std::move(loadedGame->enemies);
            spellDeck = std::move(loadedGame->spellDeck);

            // Восстанавливаем игровое поле
            field.setCellType(player.getPosition(), Cell::Type::PLAYER);
            for (const auto& enemy : enemies) {
                if (enemy->isAlive()) {
                    field.setCellType(enemy->getPosition(),
Cell::Type::ENEMY);

```

```

        }
    }

    updateTargetsList();
    std::cout << "Игра успешно загружена!\n";
}

} catch (const LoadException& e) {
    std::cerr << "Не удалось загрузить игру: " << e.what() <<
std::endl;
}
}

```

5. Обработка исключительных ситуаций

1) Типы обрабатываемых исключений

Тип исключения	Ситуация возникновения
FileAccessException	Невозможно открыть файл для записи/чтения
CorruptedSaveException	Файл сохранения поврежден или имеет неверный формат
LoadException	Общая ошибка при загрузке
SaveException	Общая ошибка при сохранении

2) Пример обработки в главном меню

```

try {
    SaveManager saver("saves/" + saves[saveChoice - 1]);
    game = saver.loadGame();
    std::cout << "Игра загружена!\n";
} catch (const LoadException& e) {
    std::cerr << "Ошибка загрузки: " << e.what() << std::endl;
    std::cout << "Начинаем новую игру...\n";
    game = std::make_unique<GameSession>(15, 15, 5);
}

```

3) Информативность сообщений

Все исключения содержат информацию о том, где и почему произошла ошибка:

- "Не удалось открыть файл: saves/game.dat"
- "ФАЙЛ ПОВРЕЖДЕН: saves/corrupted.dat"
- "ОШИБКА ЗАГРУЗКИ: Неверный формат данных"

6. Принципы ООП, примененные в работе

1) Инкапсуляция

Каждый класс скрывает детали своей реализации:

- SaveManager инкапсулирует работу с файлами и сериализацию
- TurnManager скрывает логику смены фаз
- GameSession объединяет все компоненты игры

2) Наследование

Иерархия исключений демонстрирует преимущества наследования:

- Базовый класс GameException для всех игровых ошибок
- Специализированные классы для конкретных ситуаций
- Единый интерфейс через метод what()

3) Полиморфизм

```
try {
    // код
} catch (const GameException& e) {
    // Обрабатываем любую игровую ошибку единообразно
    std::cerr << e.what() << std::endl;
}
```

4) RAII (Resource Acquisition Is Initialization)

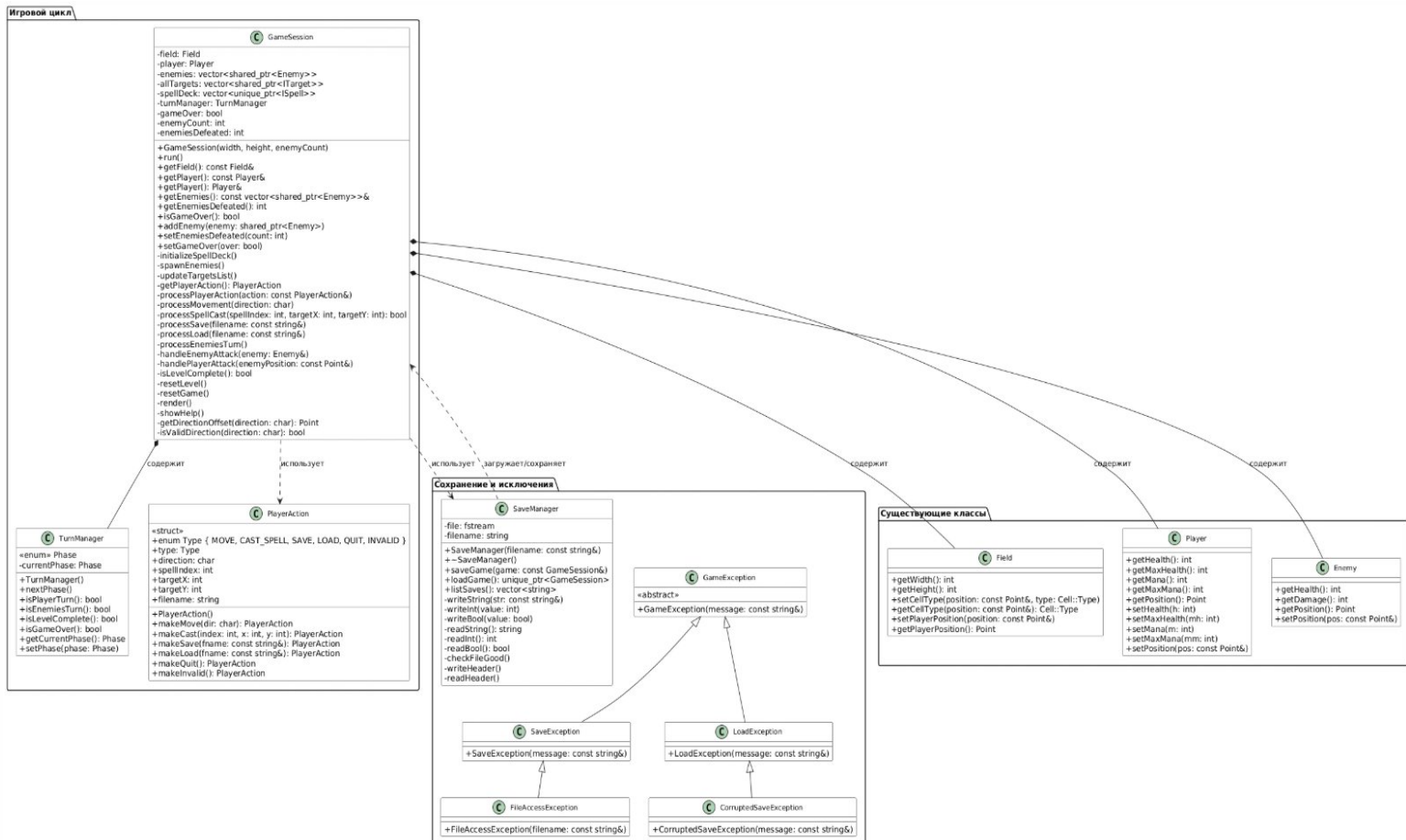
Класс SaveManager реализует идиому RAII:

- Файл открывается в конструкторе
- Файл автоматически закрывается в деструкторе
- Запрет копирования предотвращает дублирование ресурсов
- Перемещение разрешено для передачи владения

5) Композиция

GameSession содержит объекты Field, Player, TurnManager и коллекции врагов и заклинаний, демонстрируя принцип композиции.

7. UML-диаграмма классов



ЗАКЛЮЧЕНИЕ

В ходе выполнения третьей лабораторной работы я расширил свою игру, добавив полноценный игровой цикл с правильной очередностью ходов. Теперь игра работает так, как и задумывалось: сначала ходит игрок, потом враги.

Появление сохранения и загрузки, при этом сохраняется всё: позиция игрока и врагов, здоровье и мана, заклинания в руке. Теперь можно прерваться в любой момент и потом продолжить с того же места.

Появилось обработки ошибок. Раньше если что-то шло не так, программа просто падала. Теперь она понимает разные ситуации: нет файла с сохранением, файл повреждён, нельзя записать в папку. В каждом случае выводится сообщение, и игра продолжает работать. Была применена иерархия исключений — для каждого типа ошибки свой класс, но все они наследуются от общего предка.

Идиому RAII использовал в классе `SaveManager` — файл открывается в конструкторе и автоматически закрывается в деструкторе. Это гарантирует, что даже если случится ошибка, файл не останется открытым.

Игра стала полноценной. Можно ходить, атаковать, использовать заклинания, побеждать врагов и быть побежденным.